

UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

APROXIMAREA ECHILIBRULUI NASH ÎN JOCUL DE POKER HEADS-UP LIMIT HOLD'EM

Absolvent

Muscalu David-Cristian

Coordonatori științifici

Conf. Dr. Bogdan Alexe

Asist. Drd. Alexandra Diaconu

București, septembrie 2026

Rezumat

Lucrarea descrie proiectarea, implementarea și evaluarea unui bot de poker Texas Hold'em Limit, capabil să joace împotriva unui adversar uman printr-o interfață web, folosind algoritmul Monte Carlo Counterfactual Regret Minimization (MCCFR). Sistemul include un modul de antrenare paralelizat pe mai multe procese, precum și un modul independent de asistență bazat pe vedere artificială, care citește starea jocului direct de pe ecran și face o recomandare.

Abstract

This thesis describes the design, implementation and evaluation of a Limit Texas Hold'em poker bot, capable of playing against a human opponent through a web interface, using the Monte Carlo Counterfactual Regret Minimization (MCCFR) algorithm. The system includes a training module parallelized across multiple processes, as well as an independent advisory module based on artificial vision, which reads the game state directly from the screen and makes a recommendation.

Cuprins

1 Introducere 5

1.1 Context 5

1.2 Definirea problemei 5

1.3 Preliminarii 6

2 Context teoretic 7

2.1 Regret minimization și echilibrul Nash 7

2.2 Arborele de joc 8

2.3 Monte Carlo CFR (MCCFR) 9

2.4 Abstractizare: necesitate și riscuri 10

2.5 Instrumente folosite 10

3 Arhitectura sistemului 11

3.1 Prezentare generală 11

3.2 Interfața REST a aplicației de joc 11

3.3 Identificatorul de set de informație: format și partajare a logicii 12

3.4 Reconstrucția stării la interogare 13

4 Antrenarea botului MCCFR 14

4.1 Mediul și bugetul de antrenare 14

4.2 Abstractizarea stărilor de joc 14

4.3 Reprezentarea internă a identificatorului de stare 15

4.4 Paralelizarea antrenării 16

5 Modulul de asistență prin vedere artificială 17

5.1 Motivație și scenariu de utilizare 17

5.2 Arhitectura pe două procese 17

5.3 Calibrarea regiunilor de ecran 18

5.4 Recunoașterea cărților 18

5.5 Citirea istoricului de pariere 19

5.6 Reconstrucția stării de joc și verificarea de consistență 19

5.7 Bucla principală și notificarea utilizatorului 19

5.8 Exemplul unei recomandări 20

6 Evaluare experimentală 21

6.1 Metodologie 21

6.2 Rezultate: partide automate contra botului random 21

6.3 Rezultate: partide automate contra botului expectimax 22

7 Concluzii 23

Referințe 24

1 Introducere

1.1 Context

Texas Hold'em este un joc cu informație imperfectă, fiecare jucător cunoaște doar propriile cărți de start, nu și pe cele ale adversarului, iar deciziile se iau succesiv, pe parcursul mai multor runde de pariere (preflop, flop, turn, river), pe măsură ce dealerul dezvăluie cărți comune. Această caracteristică face ca teoria clasică a jocurilor cu informație perfectă (rezolvabile prin căutare completă, precum șahul) să nu se aplice. Strategia optimă a unui jucător depinde de distribuția de probabilitate pe care adversarul o are asupra propriei mâini, nu de o singură stare cunoscută a jocului.

Din punct de vedere al teoriei jocurilor, varianta cu doi jucători a pokerului Limit Hold'em este considerată rezolvată în momentul în care se ajunge la echilibrul Nash (o pereche de strategii de la care niciun jucător nu are interesul să se abată). Pentru variante suficient de mici ale jocului, un asemenea echilibru poate fi aproximat algoritmic.

1.2 Definirea problemei

Această lucrare își propune dezvoltarea unui bot capabil să joace Limit Texas Hold'em heads-up (doi jucători) la un nivel rezonabil de competent, pornind de la algoritmul Monte Carlo Counterfactual Regret Minimization (MCCFR), și integrarea sa într-o aplicație. Obiectivele concrete au fost:

- implementarea unui pipeline de antrenare MCCFR, care poate rula pe hardware obișnuit;
- proiectarea unei abstractizări a spațiului de stări (bucketizare) suficient de restrânse încât antrenarea să conveargă într-un timp rezonabil, dar destul de detaliate încât strategia să rămână utilă;
- construirea unei aplicații client-server prin care un utilizator uman poate juca efectiv împotriva botului antrenat;

- construirea unui modul independent, bazat pe vedere artificială, care să poată recomanda o acțiune unui jucător uman aflat în fața interfeței de joc.

1.3 Preliminarii

Varianta de joc este Limit Texas Hold'em heads-up, cu blinduri fixe (50/100), patru runde de pariere (preflop, flop, turn, river), dimensiuni de pariere fixe pe rundă (100 preflop și flop, 200 turn și river) și un număr maxim de 3 măriri per rundă. Fiecare jucător primește două cărți proprii iar pe masă sunt dezvăluite succesiv 3 cărți la flop, apoi câte una la turn și la river.

Am folosit biblioteca OpenSpiel [3], care oferă o implementare a jocului universal_poker ce poate fi configurată prin parametri (numărul de runde, dimensiunile pariurilor, numărul de măriri). Evaluarea forței mâinilor de poker, adică determinarea mâinii câștigătoare într-o anumită stare a jocului, este realizată cu biblioteca treys [2].

Restul lucrării este organizat în jurul a patru componente: arhitectura aplicației client-server prin care un jucător uman poate înfrunta botul antrenat (Capitolul 3), procesul de antrenare offline a strategiei (Capitolul 4), modulul de asistență prin vedere artificială (Capitolul 5) și, în final, evaluarea experimentală a botului (Capitolul 6).

2 Context teoretic

2.1 Regret minimization și echilibrul Nash

Counterfactual Regret Minimization (CFR) [7] este un algoritm iterativ care converge către un echilibru Nash aproximativ în jocurile de sumă nulă pentru doi jucători. Algoritmul realizează acest lucru prin minimizarea regretului acumulat la nivelul fiecărui set de informație. Acesta grupează stările pe care jucătorul nu le poate deosebi între ele, adică situațiile în care el are aceleași cărți în mână și a văzut aceleași acțiuni la masă, fără să știe cărțile adversarului.

La fiecare iterație t, pentru fiecare set de informație I și fiecare acțiune a posibilă în I, se calculează regretul instantaneu $r^t(I, a)$. Acesta indică câștigul suplimentar dacă se alegea mereu acțiunea a în loc de strategia curentă. Regretul cumulat este:

$$R^T(I, a) = \sum_{t=1}^{T} r^t(I, a) \tag{1}$$

iar strategia pentru iterația următoare se calculeaza prin regret matching: probabilitatea fiecărei acțiuni este proporțională cu regretul pozitiv acumulat pentru aceasta.

$$\sigma^{T+1}(I, a) = \frac{\max(R^T(I, a), 0)}{\sum_{a'} \max(R^T(I, a'), 0)} \tag{2}$$

De exemplu, presupunem că la un set de informație I jucătorul are trei acțiuni legale, FOLD, CALL și BET, iar după T iterații regretele cumulate (ecuația 1) sunt $R^T(I, \text{FOLD}) = -5$, $R^T(I, \text{CALL}) = 10$ și $R^T(I, \text{BET}) = 2$ (regretul negativ arată că FOLD a fost, în medie, o alegere mai proastă decât strategia jucată până atunci). Conform ecuației 2, doar regretele pozitive contribuie la strategie: $\max(-5, 0) = 0$, $\max(10, 0) = 10$ și $\max(2, 0) = 2$, cu suma 12. Strategia pentru iterația $T{+}1$ devine $\sigma^{T+1}(I, \text{FOLD}) = 0$, $\sigma^{T+1}(I, \text{CALL}) = 10/12 \approx 0{,}83$ și $\sigma^{T+1}(I, \text{BET}) = 2/12 \approx 0{,}17$.

Dacă nicio mutare nu este mai profitabila decât restul, algoritmul alege la întâmplare, împărțind șansele în mod egal. Strategia perfectă nu este cea la care ajunge la ultimul meci, deoarece își schimbă deciziile de la o rundă la alta. Strategia cu adevărat imba-

tabilă se obține făcând media tuturor deciziilor pe care le-a luat de-a lungul întregului antrenament.

2.2 Arborele de joc

MCCFR operează pe arborele de joc în formă extensivă al variantei de Limit Hold'em. Rădăcina arborelui corespunde începutului unei mâini și celălalte noduri sunt de unul din trei tipuri:

- nod de șansă: dealerul distribuie cărți, asta inseamnă cărțile private ale fiecărui jucător la începutul mâinii, cărțile comune la flop, turn și river. Fiecare muchie care pleacă dintr-un asemenea nod corespunde uneia dintre cărțile rămase în pachet, aleasă cu probabilitate uniformă;
- nod de decizie: este rândul unui jucător să acționeze. Muchiile care pleacă din el corespund acțiunilor legale în acel moment (FOLD, CALL/CHECK sau BET/RAISE);
- nod terminal (frunză): mâna s-a încheiat, fie printr-un FOLD, fie prin showdown la finalul rundei de river. Fiecărui nod terminal îi este asociat un payoff, adică câștigul sau pierderea netă, în jetoane, pentru fiecare jucător.

Figura 1 ilustrează un fragment al acestui arbore, mai exact runda de pariere preflop. P0 este jucătorul aflat la small blindă (SB), care pariază obligatoriu jumătate din miza minimă înainte de a se distribui cărțile și acționează primul la preflop, iar P1 este jucătorul aflat la big blindă (BB), care pariază miza minimă întreagă și acționează ultimul la preflop. Restul mâinii (flop, turn, river) continuă după același tipar până la un nod terminal.

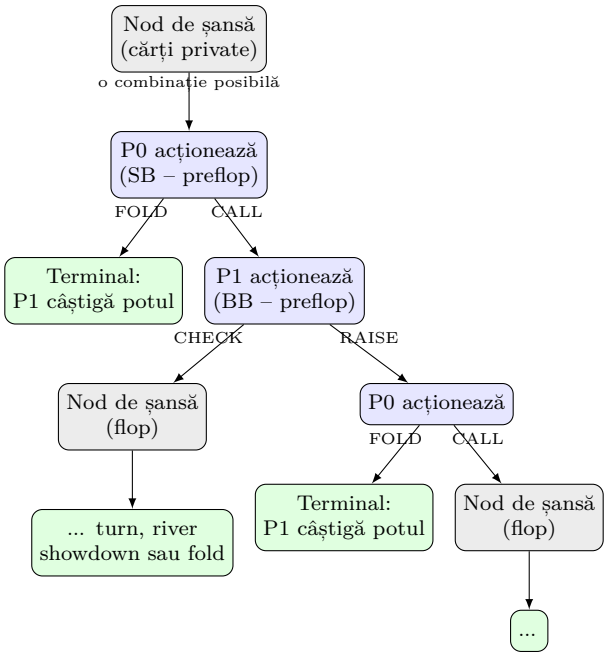


Figura 1: Fragment din arborele de joc pentru preflop. noduri de șansă (gri), noduri de decizie (albastru) și noduri terminale (verde).

2.3 Monte Carlo CFR (MCCFR)

CFR clasic [7] necesită parcurgerea completă a arborelui de joc definit mai sus la fiecare iterație, ceea ce este imposibil pentru un joc de dimensiunea Limit Hold'em pe un hardware obișnuit. Monte Carlo CFR (MCCFR) [4] rezolvă această problemă prin explorarea unei singure ramuri a arborelui la fiecare iterație, păstrând o estimare precisă a regretului. În varianta cu eșantionare externă (external sampling), o iterație constă din câte un episod per jucător. La nodurile de șansă (distribuirea cărților) și la nodurile jucătorului advers, se alege o singură acțiune conform strategiei curente iar la nodurile jucătorului pentru care se actualizează regretul, se parcurg toate acțiunile legale. Acest compromis (se analizeaza toate opțiunile jucatorului, dar o singură mutare a adversarului) păstrează calculele corecte în medie și face rundele de antrenament considerabil mai rapide.

Implementarea folosită este cea din OpenSpiel (external_sampling_mccfr) [3], care stochează fiecare situație de decizie sub forma unui text mic și pentru fiecare astfel de șir păstrează și actualizează constant două liste de numere. Aceste liste reprezintă regretele adunate (se folosesc în timpul antrenării) și strategia medie (aceasta e strategia finală

care este folosită în timpul unui joc).

2.4 Abstractizare: necesitate și riscuri

Numărul de seturi de informație în Limit Hold'em heads-up neabstractizat este combinația dintre $\binom{52}{2}$ mâini posibile, toate distribuțiile de cărți comune și toate secvențele de pariere. Practica standard este bucketizarea, adică gruparea stărilor de joc „similare" sub un identificator comun, astfel încât algoritmul să învețe o singură strategie pentru întreaga grupă în loc de o strategie separată pentru fiecare stare exactă, care este oricum imposibil de antrenat suficient.

Identificatorul folosit în această lucrare are forma

P:{jucător}|B:{bucket}|R:{rundă}|H:{istoric pariere}|L:{acțiuni legale}

unde bucket este un întreg ce reprezintă clasa de echivalență a mâinii curente (detaliată în secțiunea 4.2), iar istoric pariere este ultima secvență de patru acțiuni (fold/call/bet) din mâna curentă.

O bucketizare greșită, adica se grupează stări cu forțe de mână foarte diferite sub același identificator, sau care separă artificial stări echivalente strategic, nu produce o eroare de execuție: algoritmul rulează, converge (către un echilibru al jocului abstractizat, nu al jocului real) și produce o strategie sintactic validă. Singurul simptom vizibil este o strategie care joacă prost sau incoerent, motiv pentru care evaluarea din Capitolul 6 nu se limitează la verificarea faptului că antrenarea rulează fără erori, ci analizează cantitativ rezultatul ei, ca metodă de a distinge o strategie insuficient antrenată de o problemă structurală de abstractizare.

2.5 Instrumente folosite

Pe lângă OpenSpiel [3] și treys [2], aplicația client-server folosește FastAPI [5] și o interfață React pentru partea de client. Pentru procesarea imaginilor și recunoașterea textului se utilizează OpenCV [1], respectiv Tesseract [6].

3 Arhitectura sistemului

3.1 Prezentare generală

Sistemul este format din patru componente, care rulează independent și comunică prin HTTP:

- un backend (FastAPI [5]), care menține starea unei partide (obiectul pyspiel.State curent) și expune două rute: /api/new-game, care inițializează o mână nouă, și /api/action, care aplică acțiunea utilizatorului, apoi rulează botul până când este din nou rândul jucătorului uman sau mâna se termină;
- un frontend scris în React, care afișează masa de joc (cărți, pot, stive, istoric) și trimite acțiunile utilizatorului către backend;
- modulul MCCFR_bot, care încarcă strategia antrenată dintr-un fișier și, pentru o stare de joc dată, calculează identificatorul și alege o acțiune din cele legale conform strategiei;
- modulul de antrenare (Capitolul 4), care rulează offline, independent de celălalte componente, și produce fișierul de strategie folosit de MCCFR_bot.

Figura 2 arată interfața frontend-ului: masa de joc, cărțile jucătorului și cele comune, stivele, pot-ul, panoul LIVE LOG cu istoricul acțiunilor și butoanele de acțiune disponibile la momentul respectiv.

3.2 Interfața REST a aplicației de joc

Comunicarea dintre frontend și backend se face prin doar două rute HTTP (Tabelul 1): una pentru a începe o mână nouă și una pentru a trimite acțiunea aleasă de jucător. Ambele întorc același format de răspuns — o descriere completă a stării mesei — astfel încât frontend-ul poate redesena întreaga masă de joc pornind de la orice răspuns, indiferent de ruta apelată.

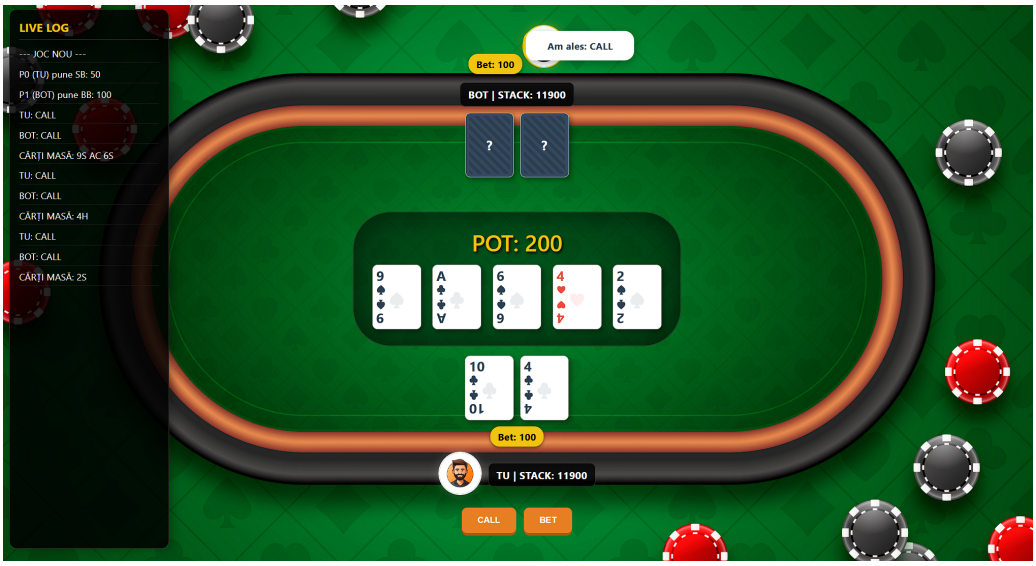


Figura 2: Interfața frontend-ului aplicației de joc, cu masa curentă, mâna jucătorului, board-ul, panoul LIVE LOG și butoanele de acțiune.

Rută	Metodă	Corp cerere
/api/new-game	POST	— (inițializează o mână nouă)
/api/action	POST	{”action_name”: ”FOLD” | ”CALL” | ”BET”}

Tabela 1: Rutele expuse de backend-ul aplicației de joc.

Răspunsul conține tot ce are nevoie interfața pentru a afișa masa: cărțile jucătorului (player_hand) și cele de pe masă (board_cards, sub forma unor șiruri precum ”Ah”), acțiunile disponibile în acel moment, pot-ul, stivele, mizele curente, deciziile luate anterior afișate pe ecran (history) și ultima acțiune a botului.

La /api/action, backend-ul nu are încredere oarbă în acțiunea primită, o verifică întâi față de acțiunile legale în starea curentă a jocului (state.legal_actions()), o aplică, apoi lasă botul să joace în buclă, atâta timp cât e rândul lui și mâna nu s-a terminat. Abia după ce se ajunge din nou la rândul jucătorului uman (sau mâna se încheie) se trimite răspunsul.

3.3 Identificatorul de set de informație: format și partajare a logicii

Identificatorul folosit are forma concretă (exemplu real, pentru un jucător cu mâna A9 la preflop, fără nicio acțiune încă):

P:0|B:34|R:0|H:[]|L:[0,1,2]

unde P este jucătorul, B bucket-ul mâinii, R indexul rundei (0 = preflop, 1 = flop, 2 = turn, 3 = river), H istoricul de pariere (aici gol deoarece nimeni nu a trebuit sa ia o decizie înca) și L acțiunile legale, separate prin virgulă, în ordinea în care le întoarce pyspiel. Ultimul câmp nu este doar informativ: la îmbinarea strategiilor mai multor procese de antrenare (secțiunea 4), acțiunile legale trebuie recuperate din cheie pentru a ști cărei acțiuni îi corespunde fiecare poziție din vectorul de strategie medie.

Atât backend-ul, cât și modulul de antrenare trebuie să calculeze exact același identificator pentru o stare dată, altfel, o strategie învățată la antrenare nu mai poate fi găsită cand este nevoie de ea.

3.4 Reconstrucția stării la interogare

Deoarece starea este citită direct din imagini, modulul de vedere computațională (Capitolul 5) nu primește automat un obiect pyspiel.State. Însă acest obiect este obligatoriu pentru a interoga biblioteca OpenSpiel, pentru a rezolva această problemă, un serviciu HTTP local reconstituie obiectul de stare și expune o singură rută:

Rută	POST /hint
Cerere	{”my_hole_cards”: [”Ah”,”Kd”], ”board_cards”: [”2c”,”7d”,”9s”], ”action_log”: [[”user”,”CALL”], [”bot”,”BET”]]}
Răspuns	{”recommended_action”: ”BET”}

Reconstrucția stării pornește de la zero și simulează cronologic mâna curentă. La nodurile de șansă sunt distribuite succesiv cărțile proprii, două cărți oarecare pentru adversar (este corect deoarece deciziile unui jucător nu trebuie influențate de cărtile ascunse ale celuilalt) și cărțile comune de pe masă. La nodurile de decizie se aplică acțiunile extrase din live log, identificate după nume (state.action_to_string) și nu prin indecși numerici. Ca mecanism de siguranță împotriva erorilor de OCR, algoritmul verifică la fiecare pas dacă jucătorul dedus din live log corespunde exact cu jucătorul desemnat de starea internă.

4 Antrenarea botului MCCFR

4.1 Mediul și bugetul de antrenare

Antrenarea a rulat în subsistemul Linux pentru Windows (WSL2), nu direct pe Windows, întrucât distribuția open_spiel pentru Linux este cea în care jocul universal_poker este disponibil (motiv detaliat și în secțiunea 5, unde aceeași constrângere determină arhitectura modulului de asistență prin vedere artificială). Sesiunea de antrenare a folosit 4 procese worker rulând simultan, iar fiecare proces worker poate ajunge la câțiva gigaocteți în memorie pe durate lungi de rulare, iar un număr mai mare de procese simultane a depășit memoria disponibilă a mașinii virtuale WSL2, ducând la terminarea forțată a proceselor de către sistemul de operare (OOM killer). Bugetul de timp alocat unei sesiuni a fost de 20 de ore — sub pragul de o zi calendaristică, cu o marjă de siguranță pentru eventuale întreruperi neplanificate — iar frecvența de îmbinare și salvare a strategiei a fost fixată la 10 minute.

4.2 Abstractizarea stărilor de joc

Bucket-ul asociat unei mâini depinde de runda curentă:

- Preflop: cele două cărți proprii sunt atribuite la unul dintre cele 169 de bucket-uri pe preflop (13 perechi, 78 de combinații de aceeași culoare și 78 de culori diferite). Maparea se face după numărul celor două cărți și relația lor de suit (aceeași culoare sau nu), independent de culorile exacte. Toate combinațiile de culori ale aceleiași mâini sunt tratate identic, deoarece au aceeași forță relativă, exemplu: 6 de romb si 6 de trefla sunt tratate exact la fel ca 6 de inima rosie si 6 de inima neagra;

- Flop, turn, river: bucket-ul este $\lfloor \text{EHS} \times m \rfloor$, unde EHS (Effective Hand Strength) este o estimare a probabilității de câștig la showdown, iar $m \in \{299, 999\}$ este numărul de bucket-uri (mai multe la river, ultima rundă, unde precizia contează mai mult).

Pentru flop, EHS-ul fiecărei mâini este obținut printr-o căutare $O(1)$ într-un tabel pre-

calculat offline, indexat după flop și după clasa canonică a mâinii (ex. "AKs"), construit prin simulare Monte Carlo cu 1000 de eșantioane pentru fiecare din cele $\binom{52}{3} = 22\,100$ combinații de flop posibile și fiecare din cele 169 de clase de mână. Pentru turn și river, numărul de configurații posibile ale mesei este mult prea mare pentru un tabel precalculat, așa că EHS este estimat la interogare: din cele 52 de cărți se elimină cele deja cunoscute (mâna proprie și cărțile comune vizibile), iar din pachetul rămas se extrag repetat, la întâmplare, câte două cărți pentru adversar și restul cărților comune necesare pentru a completa o masă de 5 cărți. Pentru fiecare din cele 100 de repetiții, mâna proprie și cea a adversarului (arbitrară, dar validă) sunt evaluate și comparate cu treys [2], iar EHS este proporția de eșantioane câștigate (egalitățile contează jumătate).

Identificatorul complet al unui set de informație include, pe lângă bucket și runda curentă, și ultimele 4 acțiuni de pariere din mâna curentă (indiferent de câte au avut loc efectiv până în acel punct). Această fereastră mărginită de istoric este potrivită deoarece păstrarea întregului istoric al mâinii, pe toate cele patru străzi, ar produce un număr de identificatori distincți mult prea mare pentru bugetul de calcul disponibil (fiecare secvență diferită de acțiuni ar genera un set de informație separat), în timp ce ultimele câteva acțiuni captează majoritatea informației relevante despre dinamica curentă a pariurilor (cine a inițiat agresiunea, câte măriri au avut loc recent).

4.3 Reprezentarea internă a identificatorului de stare

La fiecare nod vizitat în timpul unei iterații MCCFR, OpenSpiel apelează o funcție de observare pentru a obține identificatorul textual al setului de informație curent. Deoarece această funcție este executată de un număr uriaș de ori, reconstrucția stării de joc pas cu pas ar fi mult prea lentă. Pentru a optimiza procesul, identificatorul este extras direct din vectorul de întregi returnat de state.history(), care conține istoricul complet al acțiunilor și nodurilor de șansă. Primele patru valori reprezintă întotdeauna cărțile de start ale celor doi jucători, câte două pentru fiecare. Următoarele elemente sunt clasificate euristic:

- codurile sub 4 reprezintă acțiuni de pariere (fold, call, bet);
- codurile de la 4 în sus corespund cărților comune distribuite pe masă.

Delimitarea rundelor se bazează pe numărul de acțiuni și cărți observate până în acel punct. Spre exemplu, cele trei cărți de pe flop sunt așteptate doar după finalizarea acțiunilor de pe preflop. Identificatorul final este apoi asamblat din aceste date (jucător, bucket, rundă, ultimele acțiuni și opțiuni legale). În final, pentru a evita recalcularea costisitoare a bucket-ului la fiecare revizitare a unei stări, rezultatul este salvat într-un cache indexat după tuplul (mână, board, rundă, acțiuni legale, istoric).

4.4 Paralelizarea antrenării

Implementarea external_sampling_mccfr din OpenSpiel [3] rulează pe un singur fir de execuție (single-thread), nu folosește mai multe nuclee ale procesorului. Ca să fie folosit tot procesorul disponibil, antrenarea a fost paralelizată la nivel de proces, adică pornesc mai multe procese worker complet independente, fiecare cu propriul solver MCCFR și propriul fișier de checkpoint, fără să comunice între ele în timpul rulării. La pornire, fiecare proces își reinițializează explicit generatorul de numere aleatoare cu o nouă sămânță (seed). Măsura este esențială pe Linux, unde modulul multiprocessing folosește fork() și copiază starea părintelui, ceea ce ar face ca toate procesele să genereze aceleași mâini și să anuleze beneficiul paralelizării.

Fiecare worker generează o strategie proprie pe baza unor mâini alese aleatoriu. Deoarece aceste strategii sunt doar sume matematice de decizii, valorile lor pot fi adunate direct și normalizate, obținându-se exact aceeași medie ca în cazul unui singur proces care ar fi jucat toate mâinile. Astfel, un proces coordonator combină periodic salvările tuturor workerilor într-un fișier final unic, fără a întrerupe antrenarea.

5 Modulul de asistență prin vedere artificială

5.1 Motivație și scenariu de utilizare

Pe lângă integrarea directă a botului în aplicație, s-a construit un modul independent, care recomandă o acțiune unui jucător uman. Acesta citește starea jocului direct de pe ecran, fără să primească informații de la backend sau frontend.

Arhitectura finală a fost dictată de o limitare tehnică: versiunea de open_spiel pentru Windows nu vine la pachet cu modulul universal_poker (inclus doar în distribuția de Linux pe care am antrenat modelul). Din această cauză, logica de calcul nu poate rula direct pe PC-ul care face capturile de ecran (fiind un sistem Windows), așa că am împărțit soluția în două procese separate.

5.2 Arhitectura pe două procese

Modulul a fost separat în două componente ce comunică prin HTTP local pe localhost (WSL2 expune automat porturile către Windows):

- advisor_service.py rulează în subsistemul Linux (WSL), în același mediu virtual folosit la antrenare (secțiunea 4), unde universal_poker funcționează corect. Reconstituie starea de joc din datele trimise de partea Windows (secțiunea 3) și interoghează MCCFR_bot pentru o recomandare.
- advisor_windows.py rulează nativ pe Windows, responsabil de captura de ecran, recunoașterea optică a cărților și a istoricului de pariere, și afișarea recomandării printr-o notificare a sistemului de operare.

Practic, strategia MCCFR antrenată rulează acolo unde este garantat compatibil cu motorul de joc folosit la antrenare, iar captura de ecran, OCR și notificările rulează acolo unde are acces direct la ecranul fizic.

5.3 Calibrarea regiunilor de ecran

Pentru că citirea se face direct din pixeli, modulul trebuie să știe exact unde pe ecran se află fiecare element de interes. calibrate.py este un script ce trebuie rulat o dată (sau de câte ori se redimensionează fereastra browserului). Acesta face un screenshot complet al monitorului principal și cere utilizatorului să tragă câte un dreptunghi peste fiecare element relevant – cele două cărți din mână, cele cinci sloturi de board, panoul LIVE LOG și zona butoanelor de acțiune. Coordonatele absolute rezultate sunt salvate și reîncărcate la fiecare pornire a asistentului.

Figura 3 arată acest proces în timpul selectării unui slot de board: pe captura de ecran a mesei de joc, utilizatorul trage cu mouse-ul un dreptunghi (marcat cu albastru) peste elementul curent, ale cărui coordonate absolute vor fi salvate.



Figura 3: Selectarea unei regiuni în timpul calibrării

5.4 Recunoașterea cărților

Recunoașterea cărților combină două tehnici complementare: OpenCV [1] pentru localizare și identificarea simbolului de culoare, și Tesseract [6] pentru citirea rangului.

Pentru fiecare regiune calibrată, se decide întâi dacă acolo se află o carte în funcție de luminozitatea medie a zonei. Dacă da, chenarul exact al cărții este izolat și din colțul din stânga-sus se decupează separat zona rangului și zona simbolului de culoare.

Rangul este extras cu Tesseract și simbolul de culoare este identificat prin potrivire

de contur (cv2.matchShapes) în loc de comparare pixel cu pixel. La pornire se randează cele patru simboluri Unicode standard ale culorilor (inimă, romb, treflă, pică) ca imagini, acestora li se extrage conturul exterior, iar la recunoaștere se compară conturul simbolului decupat de pe ecran cu șabloanele. Culoarea (roșu sau negru) se stabilește separat, ceea ce restrânge automat comparația fie la {inimă, romb}, fie la {treflă, pică}.

5.5 Citirea istoricului de pariere

În mod similar, istoricul de pariere este preluat prin OCR din zona de live log a interfeței. Panoul are fundal negru cu text alb, deci imaginea este mai întâi mărită și binarizată (prag Otsu), apoi inversată (text negru pe fundal alb), pentru ca Tesseract să primească textul în formatul cu care obține cele mai bune rezultate. Din textul brut se extrag liniile de forma ”TU: CALL” sau ”BOT: BET”, obținând astfel, în ordine, jucătorul și acțiunea fiecărui pas al mâinii curente.

5.6 Reconstrucția stării de joc și verificarea de consistență

Cu mâna proprie, board-ul și istoricul de acțiuni citite de pe ecran, advisor_service.py reconstituie starea internă OpenSpiel a mâinii curente. Ca să se evite deciziile greșite cauzate de erorile OCR, sistemul face o verificare automată la fiecare pas. Acesta compară jucătorul citit din log cu cel care ar trebui să fie la rând conform stării reconstruite. Dacă cele două nu se potrivesc, reconstrucția este oprită imediat și se întoarce o eroare clară, în loc să continue cu o stare posibil coruptă.

Odată reconstruită starea până la decizia curentă, dacă mâna nu s-a terminat și e rândul jucătorului uman, advisor_service.py interoghează MCCFR_bot exact ca în antrenare și întoarce acțiunea recomandată.

5.7 Bucla principală și notificarea utilizatorului

advisor_windows.py rulează într-o buclă continuă, la interval de 1,5 secunde. Acesta capturează regiunile calibrate, verifică dacă e rândul jucătorului uman și, dacă da, trimite starea citită către advisor_service.py. Pentru a evita cereri repetate identice în timp ce

jucătorul se gândește, se calculează o amprentă (hash SHA-1) a stării curente (mână, board, log) și se retrimite o cerere doar când amprenta se schimbă față de ultima recomandare afișată. Recomandarea primită este afișată printr-o notificare nativă a sistemului de operare Windows.

5.8 Exemplul unei recomandări

Figura 4 ilustrează ce vede advisor_windows.py în urma citirii repetate a ecranului (interfața de joc din browser este arătată în Figura 2, secțiunea 3). Pentru fiecare stare nouă, valorile extrase prin OCR (mână, board, dacă e rândul jucătorului, istoricul de acțiuni) sunt afișate în consolă, împreună cu recomandarea primită de la advisor_service.py și cu notificarea Windows corespunzătoare.



Figura 4: Afișarea recomandării în consolă și prin notificare Windows la detectarea stării jocului.

6 Evaluare experimentală

6.1 Metodologie

Evaluarea strategiei antrenate s-a realizat prin jucarea impotriva a doi boti:

1. random, care alege uniform la întâmplare dintre acțiunile legale disponibile în orice stare, fără nicio noțiune de forța mâinii proprii sau a adversarului. Folosit ca prag minim, o strategie care nu ar reuși să bată consecvent un adversar complet neinformat ar indica o problemă gravă în antrenare sau în abstractizare, indiferent de rezultatul împotriva unui adversar mai puternic.

2. expectimax un advers de tip euristic, care alege acțiuni printr-o căutare pe adâncime limitată, ponderată de o estimare a forței mâinii proprii prin simulare Monte Carlo, fără nicio noțiune de echilibru sau bluff calculat.

Pentru ambii adversari de tip bot, cele două stive pornesc de la aceeași valoare (12 000) și se urmărește evoluția lor pe parcursul a câteva sute de mâini.

6.2 Rezultate: partide automate contra botului random

Tabelul 2 arată evoluția stivelor în partide MCCFR contra botului random.

Mâini jucate	Profit net MCCFR	Profit net random
100	+9 800	−9 800
200	+21 400	−21 400
300	+35 900	−35 900
400	+48 600	−48 600

Tabela 2: Profit net cumulat al fiecărui bot, în partide MCCFR contra botului random; fiecare mână pornește cu stivele resetate la 12 000.

Avantajul botului MCCFR este consistent și crește aproape liniar cu numărul de mâini jucate, fără nicio inversare de tendință pe parcursul rulării.

6.3 Rezultate: partide automate contra botului expectimax

Mâini jucate	Profit net MCCFR	Profit net expectimax
Rulare timpurie		
100	−3 400	+3 400
200	−14 800	+14 800
300	−29 100	+29 100
400	−41 700	+41 700
Rulare ulterioară, după antrenare suplimentară		
100	+6 200	−6 200
200	+11 800	−11 800
300	+19 400	−19 400
400	+27 900	−27 900
500	+34 600	−34 600

Tabela 3: Profit net cumulat al fiecărui bot, în partide MCCFR contra expectimax, în două rulări succesive de evaluare; fiecare mână pornește cu stivele resetate la 12 000.

În prima rulare, botul MCCFR pierde constant pe termen lung. Între cele două rulări, strategia a mai fost antrenată timp de aproximativ 200 de ore de rulare efectivă, desfășurate în mai multe sesiuni succesive. La o viteză măsurată de aproximativ 43 de iterații MCCFR pe secundă per proces worker, constantă indiferent de dimensiunea strategiei acumulate, cele 4 procese worker rulate în paralel însumează aproximativ 124 de milioane de iterații suplimentare. După această antrenare tendința se inversează complet, botul MCCFR conduce încă de la primele 100 de mâini, iar avansul acumulat crește constant, ajungând la +34 600 după 500 de mâini. Spre deosebire de partidele contra botului random (secțiunea anterioară), unde avansul e mai mare încă de la început, avantajul aici crește mai lent, consistent cu faptul că expectimax este un adversar semnificativ mai puternic decât o strategie aleatoare, dar tendința este clar și constant favorabilă botului MCCFR.

7 Concluzii

Această lucrare a urmărit dezvoltarea unui bot de poker Limit Texas Hold'em heads-up bazat pe MCCFR de la algoritmul de antrenare și abstractizarea spațiului de stări, până la o aplicație client-server și un modul independent de asistență prin vedere artificială. Pentru un sistem bazat pe învățare, absența erorilor de execuție nu garantează, prin ea însăși, o strategie corectă, motiv pentru care calitatea strategiei antrenate a fost evaluată explicit, prin partide automate împotriva a doi adversari de referință de complexitate diferită, nu doar prin verificarea faptului că antrenarea rulează fără erori.

Rezultatele confirmă că strategia învățată este utilă, nu doar sintactic validă, botul MCCFR obține un avantaj consistent și substanțial față de un adversar random încă din primele mâini jucate, iar, după o perioadă suplimentară de antrenare, ajunge să conducă și în fața adversarului euristic (expectimax), folosit ca referință semnificativ mai puternică. Această inversare de rezultat, direct corelată cu volumul de calcul investit, sugerează că principala limitare a strategiei, la acest stadiu, ține de bugetul de antrenare, nu de o problemă structurală a abstractizării alese.

Lucrarea arată că un sistem MCCFR complet, antrenare paralelizată pe mai multe procese, o abstractizare a spațiului de stări suficient de agresivă încât să mențină acest spațiu la o dimensiune tratabilă, o aplicație jucabilă și un modul suplimentar de asistență prin vedere artificială, complet independent de codul sursă al jocului, poate fi construit și antrenat pe hardware obișnuit, fără infrastructură de calcul specializată.

Referințe

[1] Gary Bradski, The OpenCV Library, Dr. Dobb's Journal of Software Tools, 2000.

[2] Will Deng, treys: A Poker Hand Evaluation Library, https://github.com/ihendley/treys, 2019.

[3] Marc Lanctot, Edward Lockhart, Jean-Baptiste Lespiau, Vinicius Zambaldi, Satyaki Upadhyay, Julien Pérolat, Sriram Srinivasan, Finbarr Timbers, Karl Tuyls, Shayegan Omidshafiei, Daniel Hennes, Dustin Morrill, Paul Muller, Timo Ewalds, Ryan Faulkner, János Kramár, Bart De Vylder, Brennan Saeta, James Bradbury, David Ding, Sebastian Borgeaud, Matthew Lai, Julian Schrittwieser, Thomas Anthony, Edward Hughes, Ivo Danihelka și Jonah Ryan-Davis, „OpenSpiel: A Framework for Reinforcement Learning in Games", în arXiv preprint arXiv:1908.09453 (2019).

[4] Marc Lanctot, Kevin Waugh, Martin Zinkevich și Michael Bowling, „Monte Carlo Sampling for Regret Minimization in Extensive Games", în Advances in Neural Information Processing Systems, 2009.

[5] Sebastián Ramírez, FastAPI, https://fastapi.tiangolo.com, 2018.

[6] Ray Smith, An Overview of the Tesseract OCR Engine, Ninth International Conference on Document Analysis and Recognition (ICDAR), 2007.

[7] Martin Zinkevich, Michael Johanson, Michael Bowling și Carmelo Piccione, „Regret Minimization in Games with Incomplete Information", în Advances in Neural Information Processing Systems, 2008.